

CHAPTER 6

Backpropagation Neural Net

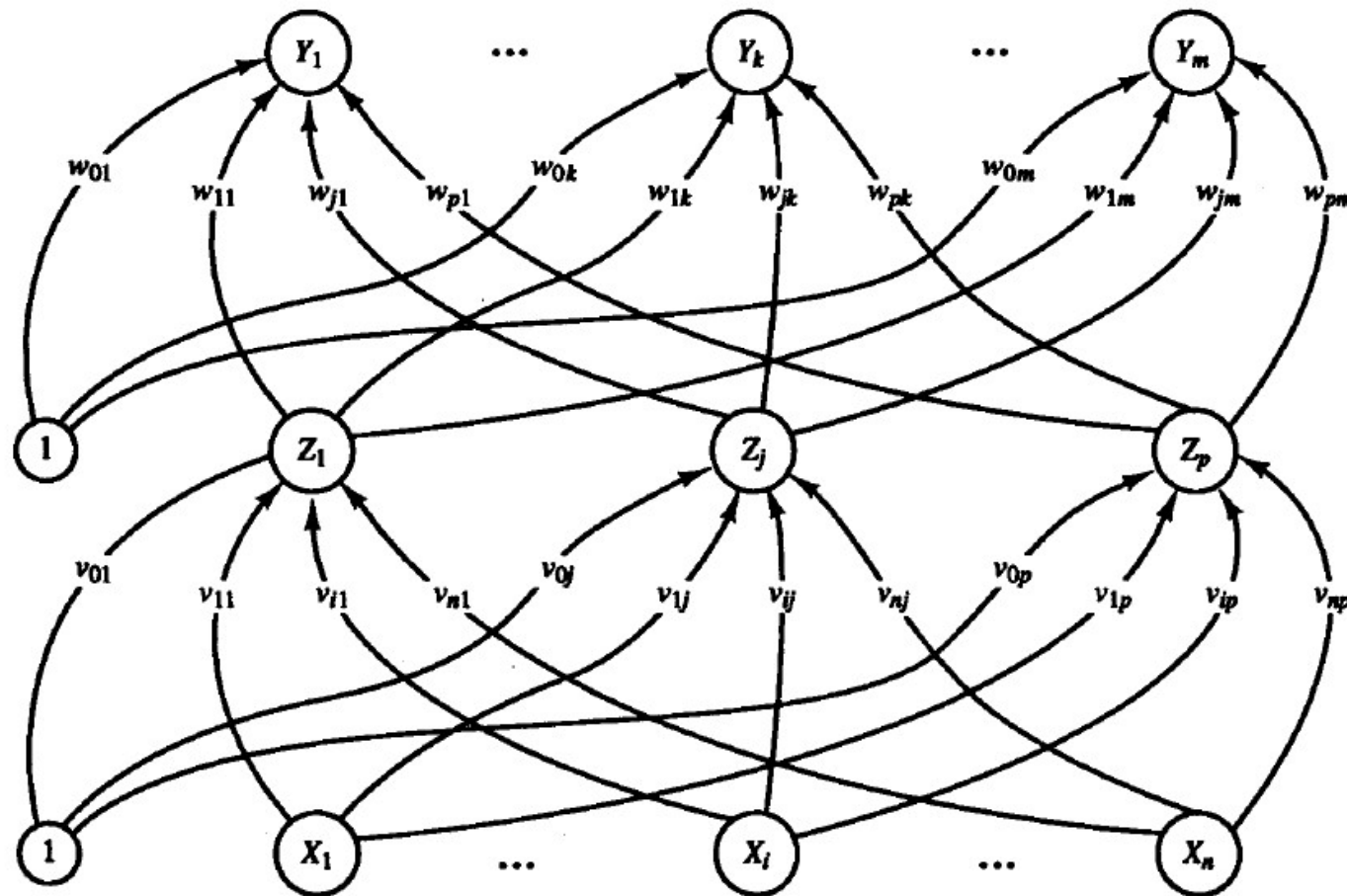
Introduction

- ❑ In this chapter, we shall discuss this training method, known as backpropagation (of errors) or the generalized delta rule.
- ❑ It is simply a **gradient descent** method to minimize the total squared error of the output computed by the net.
- ❑ The training of a network by backpropagation involves three stages: the feedforward of the input training pattern, the calculation and backpropagation of the associated error, and the adjustment of the weights .
- ❑ After training, application of the net involves only the computations of the feedforward phase.

Introduction

- ❑ Even if training is slow, a trained net can produce its output very rapidly.
- ❑ Numerous variations of backpropagation have been developed to improve the speed of the training process.
- ❑

Architecture



Architecture

- ❑ The output units (the Y units) and the hidden units also may have biases (as shown).
- ❑ The bias on a typical output unit Y_k is denoted by W_{0k} ; the bias on a typical hidden unit Z_j is denoted V_{0j} .
- ❑ These bias terms act like weights on connections from units whose output is always 1. (These units are shown in Figure 6.1 but are usually not displayed explicitly.)
- ❑ Only the direction of information flow for the feedforward phase of operation is shown.
- ❑ During the backpropagation phase of learning, signals are sent in the reverse direction.



Algorithm

- ❑ During feedforward, each input unit (X_i) receives an input signal and broadcasts this signal to each of the hidden units $Z_1, \dots, Z_p$.
- ❑ Each hidden unit then computes its activation and sends its signal (z_j) to each output unit.
- ❑ Each output unit (Y_k) computes its activation (y_k) to form the response of the net for the given input pattern.
- ❑ given input pattern.
- ❑ During training, each output unit compares its computed activation y_k with its target value t_k to determine the associated error for that pattern with that unit.

Algorithm

- ❑ Based on this error, the factor δ_k ($k = 1, \dots, m$) is computed. δ_k is used to distribute the error at output unit Y_k back to all units in the previous layer (the hidden units that are connected to Y_k).
- ❑ It is also used (later) to update the weights between the output and the hidden layer.
- ❑ In a similar manner, the factor δ_j ($j = 1, \dots, p$) is computed for each hidden unit Z_j .
- ❑ It is not necessary to propagate the error back to the input layer, but is used to update the weights between the hidden layer and the input layer

Algorithm

- ❑ Based on this error, the factor δ_k ($k = 1, \dots, m$) is computed. δ_k is used to distribute the error at output unit Y_k back to all units in the previous layer (the hidden units that are connected to Y_k).
- ❑ It is also used (later) to update the weights between the output and the hidden layer.
- ❑ In a similar manner, the factor δ_j ($j = 1, \dots, p$) is computed for each hidden unit Z_j .
- ❑ It is not necessary to propagate the error back to the input layer, but is used to update the weights between the hidden layer and the input layer

Algorithm

- ❑ After all of the δ factors have been determined, the weights for all layers are adjusted simultaneously.
- ❑ The adjustment to the weight w_{jk} (from hidden unit Z_j to output unit Y_k) is based on the factor δ_k and the activation z_j of the hidden unit Z_j .
- ❑ The adjustment to the weight v_{ij} (from input unit X_i to hidden unit Z_j) is based on the factor δ_j and the activation x_i of the input unit.
- ❑

Algorithm

x Input training vector:

$$\mathbf{x} = (x_1, \dots, x_i, \dots, x_n).$$

t Output target vector:

$$\mathbf{t} = (t_1, \dots, t_k, \dots, t_m).$$

δ_k Portion of error correction weight adjustment for w_{jk} that is due to an error at output unit Y_k ; also, the information about the error at unit Y_k that is propagated back to the hidden units that feed into unit Y_k .

δ_j Portion of error correction weight adjustment for v_{ij} that is due to the backpropagation of error information from the output layer to the hidden unit Z_j .

α Learning rate.

X_i Input unit i :

For an input unit, the input signal and output signal are the same, namely, x_i .

v_{0j} Bias on hidden unit j .

Algorithm

Z_j Hidden unit j :
The net input to Z_j is denoted z_in_j :

$$z_in_j = v_{0j} + \sum_i x_i v_{ij}.$$

The output signal (activation) of Z_j is denoted z_j :

$$z_j = f(z_in_j).$$

w_{0k} Bias on output unit k .

Y_k Output unit k :
The net input to Y_k is denoted y_in_k :

$$y_in_k = w_{0k} + \sum_j z_j w_{jk}.$$

The output signal (activation) of Y_k is denoted y_k :

$$y_k = f(y_in_k).$$

Activation function

- ❑ An activation function for a backpropagation net should have several important characteristics: It should be continuous, differentiable, and monotonically nondecreasing.
- ❑ Furthermore, for computational efficiency, it is desirable that its derivative be easy to compute.
- ❑ For the most commonly used activation functions, the value of the derivative (at a particular value of the independent variable) can be expressed in terms of the value of the function (at that value of the independent variable).
- ❑ Usually, the function is expected to saturate, i.e., approach finite maximum and -minimum values asymptotically.

Activation function

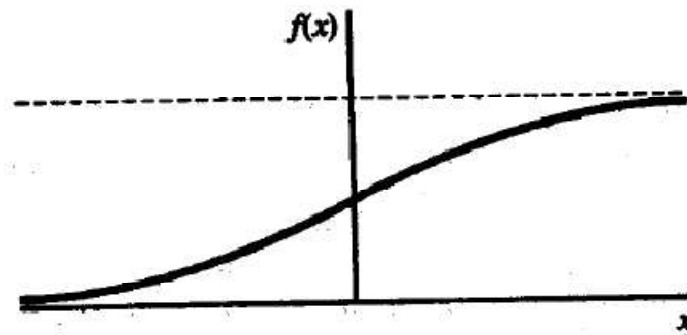


Figure 6.2 Binary sigmoid, range (0, 1).

$$f_1(x) = \frac{1}{1 + \exp(-x)},$$

$$f'_1(x) = f_1(x)[1 - f_1(x)].$$

Activation function

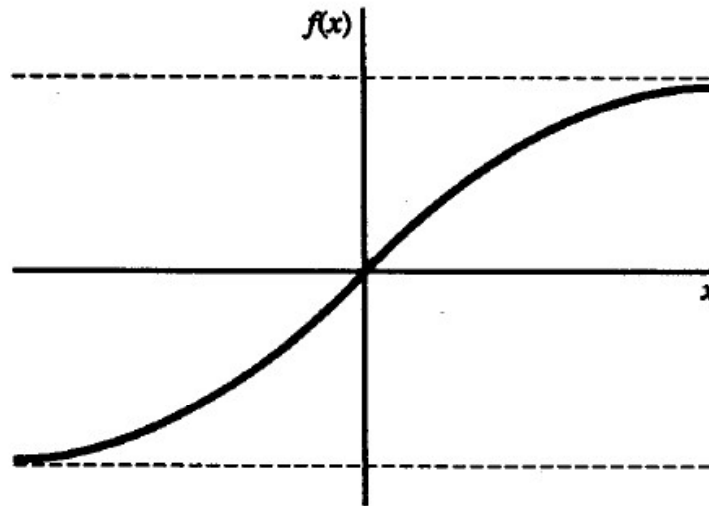


Figure 6.3 Bipolar sigmoid, range $(-1, 1)$.

$$f_2(x) = \frac{2}{1 + \exp(-x)} - 1,$$

$$f'_2(x) = \frac{1}{2} [1 + f_2(x)][1 - f_2(x)].$$

Training Algorithm

- Step 0.* Initialize weights.
(Set to small random values).
- Step 1.* While stopping condition is false, do Steps 2–9.
- Step 2.* For each training pair, do Steps 3–8.
- Feedforward:*
- Step 3.* Each input unit ($X_i, i = 1, \dots, n$) receives input signal x_i and broadcasts this signal to all units in the layer above (the hidden units).
- Step 4.* Each hidden unit ($Z_j, j = 1, \dots, p$) sums its weighted input signals,
- $$z_in_j = v_{0j} + \sum_{i=1}^n x_i v_{ij},$$
- applies its activation function to compute its output signal,
- $$z_j = f(z_in_j),$$
- and sends this signal to all units in the layer above (output units).

Training Algorithm

Step 5. Each output unit ($Y_k, k = 1, \dots, m$) sums its weighted input signals,

$$y_in_k = w_{0k} + \sum_{j=1}^p z_j w_{jk}$$

and applies its activation function to compute its output signal,

$$y_k = f(y_in_k).$$

Backpropagation of error:

Step 6. Each output unit ($Y_k, k = 1, \dots, m$) receives a target pattern corresponding to the input training pattern, computes its error information term,

$$\delta_k = (t_k - y_k)f'(y_in_k),$$

calculates its weight correction term (used to update w_{jk} later),

$$\Delta w_{jk} = \alpha \delta_k z_j,$$

calculates its bias correction term (used to update w_{0k} later),

$$\Delta w_{0k} = \alpha \delta_k,$$

Training Algorithm

Step 7. Each hidden unit ($Z_j, j = 1, \dots, p$) sums its delta inputs (from units in the layer above),

$$\delta_in_j = \sum_{k=1}^m \delta_k w_{jk},$$

multiplies by the derivative of its activation function to calculate its error information term,

$$\delta_j = \delta_in_j f'(z_in_j),$$

calculates its weight correction term (used to update v_{ij} later),

$$\Delta v_{ij} = \alpha \delta_j x_i,$$

and calculates its bias correction term (used to update v_{0j} later),

$$\Delta v_{0j} = \alpha \delta_j.$$

Training Algorithm

Update weights and biases:

Step 8. Each output unit ($Y_k, k = 1, \dots, m$) updates its bias and weights ($j = 0, \dots, p$):

$$w_{jk}(\text{new}) = w_{jk}(\text{old}) + \Delta w_{jk}.$$

Each hidden unit ($Z_j, j = 1, \dots, p$) updates its bias and weights ($i = 0, \dots, n$):

$$v_{ij}(\text{new}) = v_{ij}(\text{old}) + \Delta v_{ij}.$$

Step 9. Test stopping condition.

Training Algorithm

- ❑ An epoch is one cycle through the entire set of training vectors. Typically, many epochs are required for training a backpropagation neural net.
- ❑ The foregoing algorithm updates the weights after each training pattern is presented.
- ❑ A common variation is batch updating, in which weight updates are accumulated over an entire epoch (or some other number of presentations of patterns) before being applied.
- ❑ Note that $f'(y_{ink})$ and $f'(z_{inj})$ can be expressed in terms of y_k and z_j , respectively, using the appropriate formulas (depending on the choice of activation function)

Training Algorithm

- ❑ The mathematical basis for the backpropagation algorithm is the optimization technique known as gradient descent.
- ❑ The gradient of a function (in this case, the function is the error and the variables are the weights of the net) gives the direction in which the function increases more rapidly; the negative of the gradient gives the direction in which the function decreases most rapidly.

Random Initialization

- ❑ The choice of initial weights will influence whether the net reaches a global (or only a local) minimum of the error and, if so, how quickly it converges.
- ❑ The update of the weight between two units depends on both the derivative of the upper unit's activation function and the activation of the lower unit.
- ❑ For this reason, it is important to avoid choices of initial weights that would make it likely that either activations or derivatives of activations are zero.

Random Initialization

- ❑ The values for the initial weights must not be too large, or the initial input signals to each hidden or output unit will be likely to fall in the region where the derivative of the sigmoid function has a very small value (the so-called saturation region).
- ❑ On the other hand, if the initial weights are too small, the net input to a hidden or output unit will be close to zero, which also causes extremely slow learning.
- ❑ A common procedure is to initialize the weights (and biases) to random values between -0.5 and 0.5 (or between - 1 and 1 or some other suitable interval).

Nguyen-Widrow

- ❑ The following simple modification of the common random weight initialization just presented typically gives much faster learning.
- ❑ The approach is based on a geometrical analysis of the response of the hidden neurons to a single input; the analysis is extended to the case of several inputs by using Fourier transforms.
- ❑ Weights from the hidden units to the output units (and biases on the output units) are initialized to random values between -0.5 and 0.5, as is commonly the case.
- ❑ The initialization of the weights from the input units to the hidden units is designed to improve the ability of the hidden units to learn.

Nguyen-Widrow

n number of input units
 p number of hidden units
 β scale factor:

$$\beta = 0.7 (p)^{1/n} = 0.7 \sqrt[n]{p}$$

- ❑ The procedure consists of the following simple steps:
- ❑ for each hidden unit ($j = 1, \dots, p$):
- ❑ Initialize its weight vector (from the input units):

Nguyen-Widrow

$v_{ij}(\text{old}) = \text{random number between } -0.5 \text{ and } 0.5 \text{ (or between } -\gamma \text{ and } \gamma).$

Compute $\|\mathbf{v}_j(\text{old})\| = \sqrt{V_{1j}(\text{old})^2 + V_{2j}(\text{old})^2 + \dots + V_{nj}(\text{old})^2}$
Reinitialize weights:

$$v_{ij} = \frac{\beta v_{ij}(\text{old})}{\|\mathbf{v}_j(\text{old})\|}.$$

Set bias:

$v_{0j} = \text{random number between } -\beta \text{ and } \beta.$

How long to train

- ❑ Since the usual motivation for applying a backpropagation net is to achieve a balance between correct responses to training patterns and good responses to new input patterns (i.e., a balance between memorization and generalization), it is not necessarily advantageous to continue training until the total squared error actually reaches a minimum.
- ❑ Hecht-Nielsen (1990) suggests using two sets of data during training: a set of training patterns and a set of training-testing patterns. These two sets are disjoint.
- ❑ Weight adjustments are based on the training patterns; however, at intervals during training, the error is computed using the training-testing patterns.

How long to train

- ❑ As long as the error for the training- testing patterns decreases, training continues.
- ❑ When the error begins to increase, the net is starting to memorize the training patterns too specifically (and starting to lose its ability to generalize).
- ❑ At this point, training is terminated.

Number of training pairs

- ❑ Enough training patterns is determined by the condition

$$\frac{W}{P} = e,$$

- ❑ Or

$$P = \frac{W}{e}.$$

- ❑ For example, with $e = 0.1$, a net with 80 weights will require 800 training patterns to be assured of classifying 90% of the testing patterns correctly, assuming that the net was trained to classify 95% of the training patterns correctly.

Application procedure

- ❑ After training, a backpropagation neural net is applied by using only the feedforward phase of the training algorithm.
- ❑ The application procedure is as follows:
 - ❑ Step 0. Initialize weights (from training algorithm).
 - ❑ Step 1. For each input vector, do Steps 2-4.
 - ❑ Step 2. For $i = 1, \dots, n$: set activation of input unit x_i ;

Application procedure

Step 3. For $j = 1, \dots, p$:

$$z_in_j = v_{0j} + \sum_{i=1}^n x_i v_{ij};$$

$$z_j = f(z_in_j).$$

Step 4. For $k = 1, \dots, m$:

$$y_in_k = w_{0k} + \sum_{j=1}^p z_j w_{jk};$$

$$y_k = f(y_in_k).$$

Example 6.1

- ❑ A backprop net for the XOR function: binary representation.
- ❑ Training using binary input is relatively slow; any unit that receives an input of zero for a particular pattern cannot learn that pattern.
- ❑ For the binary case, training took almost 3,000 epochs.

Example 6.1

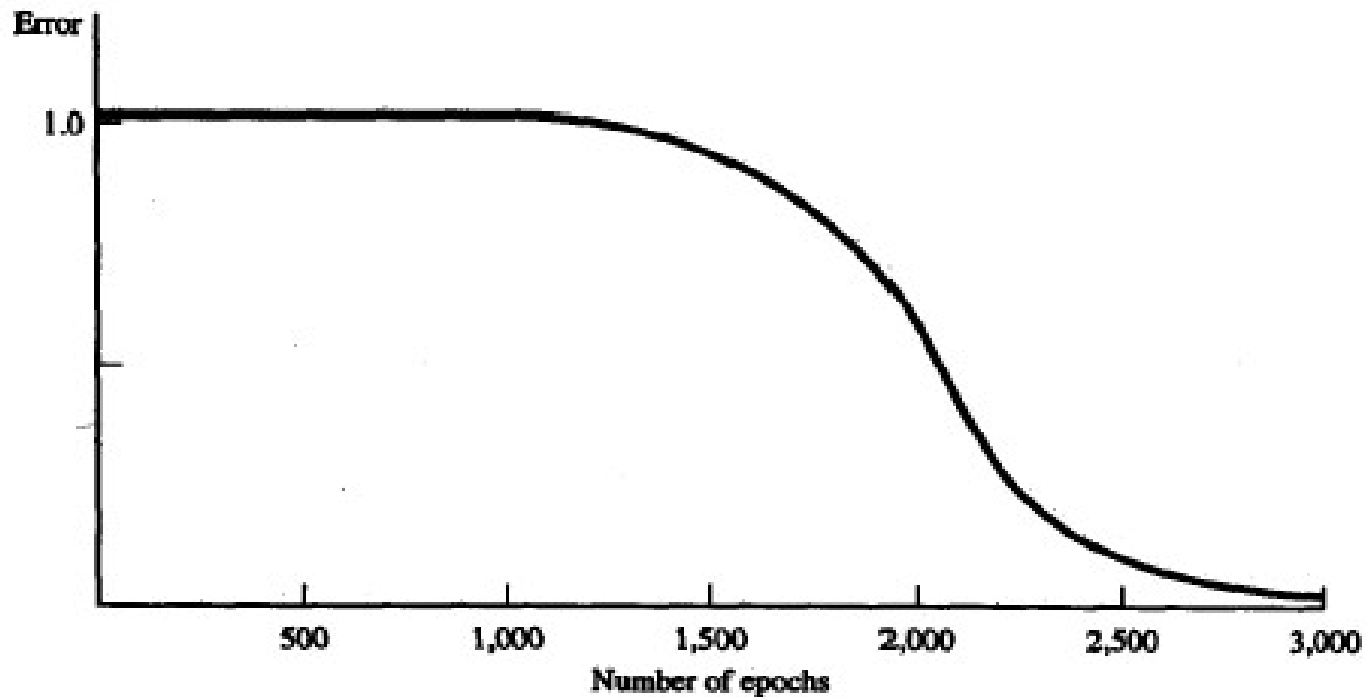
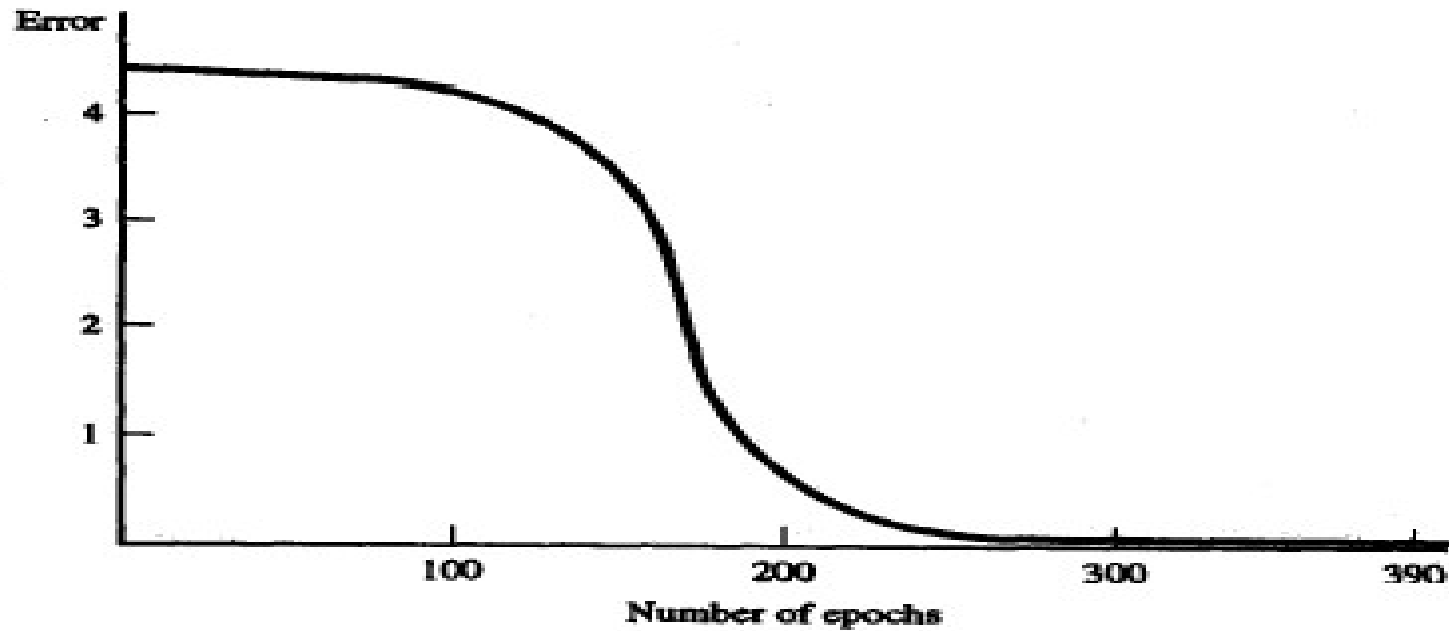


Figure 6.4 Total squared error for binary representation of XOR problem.

Example 6.2

- ❑ A backprop net for the XOR function: bipolar representation.
- ❑ Using bipolar representation for the training data for the XOR problem and the bipolar sigmoid for the activation function for the hidden and output units gives much faster training than for the binary case illustrated in Example 6.1.
- ❑ Now, training takes only 387 epochs.

Example 6.1



Example 6.3

- ❑ A backprop net for the XOR function: modified bipolar representation.
- ❑ Convergence is often improved for target values that are not at the asymptotes of the sigmoid function, since those values can never be reached.
- ❑ Using the same initial weights and bipolar training input vectors as before, but targets of +0.8 or -0.8, gives convergence (total squared error $<.05$) in 264 epochs of training.

Example 6.4

- ❑ A backprop net for the XOR function: Nguyen-Widrow weight Initialization.
- ❑ Using Nguyen-Widrow weight initialization improved the training performance of the nets considered in Examples 6.1-6.3 (binary, standard bipolar, and bipolar with targets of +0.8 and -0.8).
- ❑ The weights from the hidden units to the output units were the same in each case.
- ❑ The Nguyen-Widrow initialization for the weights to the hidden units started with the values used in the previous examples.
- ❑ The weights were scaled so that the weight vector for each hidden unit was of length $0.7\sqrt{4} = 1.4$

Example 6.4

- The biases were scaled so that they fell between - 1.4 and 1.4 (rather than between -0.5 and 0.5). The epochs required were as follows:

■	RANDOM	NGUYEN-WIDROW
Binary XOR	2,891	1,935
Bipolar XOR	387	224
Modified bipolar XOR (targets = +0.8 and -0.8)	264	127

Example 6.4

- ❑ In these examples, the use of Nguyen-Widrow initialization not only improved the training speed, but also greatly reduced the chances of generating initial weights for which the net fails to converge.
- ❑ To study the sensitivity of these - simple backpropagation nets to the choice of initial weights, eight different sets of random weights were used in the examples.
- ❑ For the binary representation, with random initial weights, only one of the eight cases converged in fewer than 3,000 epochs.

VARIATIONS

- ❑ Several modifications can be made to the backpropagation algorithm presented on page 294 which may improve its performance in some situations.
- ❑ The modifications we discuss involve changes to the weight update procedure, alternatives to the sigmoid activation functions presented previously, a variation to improve biological plausibility and computational power, and finally an explicit statement of the backpropagation algorithm for 2 hidden layers.

Momentum

- ❑ In backpropagation with momentum, the weight change is in a direction that is a combination of the current gradient and the previous gradient.
- ❑ For example, in the simplest form of backpropagation with momentum, the new weights for training step $t + 1$ are based on the weights at training steps t and $t - 1$.

Example 6.6

- ❑ A backprop net with momentum for the XOR function: bipolar representation
- ❑ Using the same initial weights and architecture as in Examples 6.1-6.3 and target values of + 1.0 and - 1.0, adding momentum (0.9) with a learning rate as before (0.2) reduces the training requirements from 387 epochs to 38 epochs.

Batch updating

- ❑ In some cases it is advantageous to accumulate the weight correction terms for several patterns (or even an entire epoch if there are not too many patterns) and make a single weight adjustment (equal to the average of the weight correction terms) for each weight rather than updating the weights after each pattern is presented.
- ❑ This procedure has a smoothing effect on the correction terms.
- ❑ In some cases, this smoothing may increase the chances of convergence to a local minimum.